

Core Platform Framework

아키텍처설계서

아키텍트 · 기술책임자 · 도입 검토자

문서 목적	CPF 의 목표 Architecture, 구현 책임 경계, Module Ownership, Topology, Transaction/Integration/Data/Operation 경계와 Trade-off 를 설명합니다.
Repository / Branch	freeangelsun/202412_01_CPF / master
기준 Source / 정보	Repository f6d7080c5a14b7dd7595093f9497470169e18d80 Product Source f0aa49f29cba3cfd6ae12b0ddd4e118d05fff16c 최상위 요구사항 cpf-docs/governance/CPF_FINAL_TARGET_REQUIREMENTS.md 문서 표준 cpf-docs/specification/CPF_DOCUMENTATION_STANDARD.md
문서 성격	완성 기준 사용자 문서. 제품이 완료된 상태에서 사용자가 설치·개발·운영·복구·검 증 업무를 끝낼 수 있도록 정상 절차, 실패 판정, 복구, 확인 근거를 기술합니다.

읽는 방법 먼저 “빠른 찾기”에서 질문을 찾고, 상세 설명은 각 기능의 정보 절로 이동하십시오. 다른 매뉴얼은 같은 내용을 반복하지 않고 정보 절을 참조합니다.

빠른 찾기 - 질문에서 정보 절로

개발자가 묻는 질문	이 문서의 정보 위치	다른 문서 연결
CPF 의 핵심 구조는?	§3 제품 구조	Core/Common/Starter/Products/Tools
왜 Local/Remote 모두 지원?	§5 Topology	업무 계약 동등성
Transaction 은 어디까지?	§7	Owner 별 Local TX
Gateway/ADM/BZA 역할은?	§4 Ownership	Control Plane/Business Admin/Trust Boundary
어떤 품질을 목표?	§2	운영/보안/복구/검증

완성 기준 이 문서는 제품 완료 상태에서 사용자가 수행할 최종 업무 계약을 기술합니다. 각 절은 선행조건 → 입력/설정 → 실행 → 정상 판정 → 실패/부분 실패 → 복구/대사 → Evidence 순서로 단으며, Source Trace 는 구현 위치를 제공합니다.

1. Architecture 결정 원칙

- 업무 Owner 와 기술 Provider 를 분리합니다.
- Public API/SPI 는 topology 와 OSS 구현에 종속되지 않게 합니다.
- 분산 환경에서 하나의 DB Transaction 으로 보이지 않게 하고 결과 불명/복구를 모델링합니다.
- 운영 Control Plane 이 업무 원장을 직접 수정하지 않습니다.
- 선택하지 않은 Capability 가 JAR/Bean/SQL/Secret 요구를 유입하지 않게 합니다.

2. 제품이 해결하려는 문제와 품질 속성

속성	Architecture 대응
확장성	Generated Domain + Public Contract + Capability Profile
운영성	ADM/관측/Runbook/상태 모델
복구 가능성	Idempotency/UNKNOWN/Reconcile/Compensation/Restart
보안/감사	Trust boundary/Permission/Approval/Masking/Audit
배포 유연성	JAR/WAR/Static/Worker/Multi-instance
검증 가능성	Source/API/SQL/Test/Config/Guide/Evidence 양방향 정합

3. 전체 제품 구조

층	Module/영역	책임
Contract	cpf-core	기술 중립 API/SPI/Context/Error/ID
Business common	cpf-common	고객 공통 Calendar/Code/정책
Capability runtime	cpf-starters/**	선택 기술 Provider/AutoConfig
Domain	Generated Domain/cpf-member	업무 원장/API/Application/Persistence
Batch	cpf-batch/**	Spring Batch/Scheduler/Worker/Agent/Center-Cut
Control Plane	cpf-admin	플랫폼 조회/제어/승인/감사
Business Admin	cpf-biz-admin	조직/권한/결재/업무 관리
Gateway	cpf-gateway	외부 trust boundary/route/attempt
Tools	cpf-tools	Generator/BOM/DB/Release/Verification

4. Module Ownership 과 의존 방향

Generated Domain → cpf-common → cpf-core

cpf-gateway → cpf-core Public Contract + 선택 Starter

cpf-batch → cpf-core Public Contract + Business Public Contract

CPF |

cpf-admin → Operations Query/Command Contract

Customer Adapter → cpf-common/core SPI

- Core 의 Common/Admin/Batch/업무 역방향 의존 금지
- 업무 Domain 간 DB 직접 접근 금지
- ADM/BZA 가 Owner DB 직접 변경 금지
- Internal Package 외부 compile 금지

5. 지원 Topology 와 동등성

Topology	핵심
Embedded Boot JAR	업무 Runtime 에 포함
External WAS WAR	Container 배포
Modular Monolith	Local Facade 로 동일 계약
Microservice/분리 WAS	Remote Facade 로 동일 계약
ADM/BZA Static Web	Backend 와 분리 배포 가능
Gateway Runtime	독립 Data Plane
Agent/Runner/Worker	독립 Batch process
Multi-instance/Multi-zone	Lease/Fencing/Shared state
Rolling/Canary/Blue-Green	Version/health/drift 통제
DR	Backup/Restore/Failover/Failback

6. Starter/Profile Architecture

계층	역할
Leaf Starter	하나의 기술 Capability/Provider
Capability Profile	업무 유형을 승인 Leaf 목록으로 해석하고 lock
Aggregate Starter	안정된 조합의 dependency 만, Bean/정책 없음
BOM	Version 정렬만

7. Transaction / Consistency Architecture

- 한 Owner 의 DB Current+History+Audit+Outbox 는 같은 Local TX 로 설계할 수 있습니다.
- HTTP/Broker/다른 Service DB/외부기관은 같은 Local TX 가 아닙니다.
- A→B→C 는 transactionId 를 유지하되 segment/attempt/trace/instance 로 계보를 분리합니다.
- Commit 후 response loss 는 rollback 가정이 아니라 UNKNOWN_RESULT + Reconcile 대상입니다.
- 이미 확정된 원격 부작용 취소는 Compensation 계약으로 다룹니다.

8. Integration Architecture

방식	제품 계약	경계
HTTP	CpfHttpClient/Service Registry	timeout/retry/idempotency
Messaging	공통 Message + Outbox/Inbox	duplicate/DLQ/replay
TCP/전문	integration capability	connection/protocol/version
File/SFTP	File/Attachment/SFTP capability	checksum/수명주기/path
Gateway	Route/Target/Auth/Attempt	외부 trust boundary

9. Data Architecture

- DB Owner 를 Module/Domain 단위로 분리하고 교차 Table update 를 금지합니다.
- Vendor Lifecycle 은 MariaDB/PostgreSQL/Oracle 의미를 맞추고 설치/verify/rollback evidence 를 분리합니다.

- Generated Domain 은 Platform DB Pack 과 독립 manifest lifecycle 을 가집니다.

10. Operations / Security Architecture

- ADM 은 상태/조치/승인/감사 Control Plane 이며 업무 DB 를 직접 수정하지 않습니다.
- BZA 는 조직/권한/결재 제품, Gateway 는 외부 trust boundary 입니다.
- Secret/PII 원문은 로그/화면/문서에 남기지 않고 service identity/permission 은 서버가 판정합니다.

11. 주요 Trade-off

선택	장점	비용/주의
Owner 별 Local TX	경계 명확/복구 가능	분산 업무는 compensation/reconcile 필요
Profile+Leaf	경량/선택 명확	Catalog/lock 관리 필요
Control Plane 분리	운영 통제/감사	Owner Contract 설계 필요
Fail-closed	보안/오배포 방지	초기 Bootstrap/운영 절차가 더 엄격

12. 완료 상태에서 유지해야 할 Architecture 불변식

- Architecture 완료 상태에서는 Public Contract, 실제 Consumer, Runtime Topology, 운영 Route, SQL/Config/Test 가 같은 Ownership 과 Failure Semantics 를 표현해야 합니다.
- Topology 가 바뀌어도 DTO/Error/Identity/Timeout/Idempotency/Audit/Recovery 의미는 동일해야 하며, 이를 깨는 구현은 Architecture 변경으로 승인받아야 합니다.

관련 정보 문서

제품 기능/도입 → [../guides/00 프레임워크안내.pdf](#) / Navigator

온라인 개발 → [../guides/01 개발자매뉴얼.pdf](#) / 개발

플랫폼 운영 → [../guides/05 플랫폼운영매뉴얼.pdf](#) / 설치/운영

Source Trace / 구현 근거

구분	실제 경로·계약	현재 판정
최상위 요구	cpf-docs/governance/CPF_FINAL_TARGET_REQUIREMENTS.md	구현 근거 확인
Build graph	settings.gradle	구현 근거 확인
Starter catalog	cpf-tools/generator/contracts/cpf-starter-catalog.json	구현 근거 확인

13. 주요 실행 흐름 Sequence

흐름	Sequence
온라인 Command	Client→Gateway(optional)→Domain API→Application→Owner DB/Outbox→Remote/Async→ADM Trace
조회	Client→Domain Query→Repository→Page/Slice/Cursor→Response
비동기	Local TX→Outbox→Publisher→Broker→Inbox/Consumer→Owner TX→DLQ/Reconcile
Batch	Scheduler/Manual→Job Execution→Step/Chunk/Partition→Worker→Metadata→ADM
Gateway Publish	ADM Draft→Validation/Approval→Gateway Instances→ACK/NACK→Current/LKG
복구	Incident/Unknown→Observation→Operation lookup→Reconcile/Compensation→Audit

14. Architecture 질문별 Decision Records

질문	결정	이유
왜 Distributed TX/XA 를 기본으로 안 쓰나?	Owner 별 Local TX + 업무 복구	외부/다중 시스템 부분 실패를 명시적으로 다룸
왜 Internal 호출도 Gateway 강제 안 하나?	Local/Remote 직접 계약	latency/결합/가용성 경계 단순화
왜 Starter 를 Leaf 로?	Capability 선택/경량 core	선택하지 않은 provider 강제 의존 방지
왜 ADM 이 DB 직접 수정 안 하나?	Owner Command 호출	업무 불변식/감사/Transaction Owner 유지
왜 UNKNOWN_RESULT 상태가 필요한가?	성공/실패 이분법 회피	Commit 후 response loss 현실 반영
왜 BZA/Gateway 선택 제품?	업무 요구가 있을 때만	모든 프로젝트 강제 복잡도 방지

15. Architecture Quality Scenario

Scenario	자극	기대 Architecture 반응
Instance crash	Command 처리 중 process kill	lease/fencing/operation 으로 중복 writer 차단 및 결과 대 사
DB failover	Primary 장애	readiness 차단→failover→schema/state 검증
Broker outage	publish 불가	업무 DB+Outbox 유지, publisher retry/DLQ
Remote response loss	Target commit 후 network loss	UNKNOWN_RESULT, blind retry 금지
Route partial apply	Gateway instance 일부 NACK	PARTIAL, LKG/reconcile
Secret rotation	credential 만료	provider rotate/reload/audit, raw log 없음

16. Module 별 Architecture Contract

Module	책임	Architecture 계약	금지
cpf-core	Public API/SPI/Context/Error/ID	기술 중립/경량	Common/Admin/Batch/업무 역의존 금지
cpf-common	고객 공통 정책	Core SPI 확장	업무 원장 난립 금지
cpf-starters	Provider/AutoConfig	선택 기능만	Mega starter/숨은 provider 금지
Generated Domain	업무 원장/API	Owner local TX	다른 Domain DB 직접 접근 금지
cpf-batch	Batch runtime/control	Spring Batch metadata	교육 업무 로직 삽입 금지
cpf-admin	Platform control plane	Owner Query/Command	업무 DB direct update 금지
cpf-biz-admin	Business admin	조직/권한/결재	고객 업무 원장 대체 금지
cpf-gateway	External trust boundary	Route/Target/Attempt	내부 호출 강제 경유 금지
cpf-tools	Generator/DB/Build/Release	정본 catalog/evidence	고정 domain 추정 금지

17. Topology 별 책임 변화 없음

관점	Modular Monolith	Microservice	Multi-instance
Business contract	Local Facade	Remote Facade	동일
Validation/Error	동일	동일	동일
Transaction	Owner local	Owner 별 local	각 instance owner
Idempotency	필요 시	Remote Command 필수 검토	shared/durable
Trace	local segment	remote child segment	instance/attempt
Auth	user context	service identity 포함	shared policy
Recovery	local rollback	reconcile/compensation	lease/fencing/reconcile

18. Data / Message / File Ownership

자산	Owner	변경 경로	복구 근거
업무 DB	Generated Domain	Application/Persistence	version/history/audit
ADM DB	cpf-admin	ADM services	operation/audit
BZA DB	cpf-biz-admin	BZA services	effective date/history
Batch Metadata	cpf-batch/Spring Batch	JobRepository/runtime	execution/checkpoint
Outbox/Inbox	업무/공통 reliability	local TX + publisher/consumer	attempt/DLQ
File artifact	업무/File capability	temp→validated→final	checksum/status
Gateway config	cpf-gateway control	versioned publish	checksum/LKG

19. Security Trust Boundary

경계	신뢰 근거	금지
Public client→Gateway	TLS/token/HMAC/audience/nonce	client header 를 service identity 로 신뢰
Gateway→Service	service identity/standard headers	외부 credential 를 그대로 내부 root 권한으로 사용
Service→Service	authenticated service identity	transactionId header 형식만 보고 신뢰
Operator→ADM	operator session/MFA/permission	menu hide 만 authorization
BZA admin	user/role/data scope	조직 범위 무시
Control Plane→Gateway	TLS/shared secret/HMAC/nonce	control endpoint public exposure

20. Deployment Architecture

Artifact	배포 단위	상태 정보	Rollback
Business service	JAR/WAR	runtime health/version	previous artifact + DB compatibility
ADM/BZA frontend	static artifact	build hash/route contract	previous static
Gateway	independent runtime	route/runtime version	LKG/config + artifact
Batch worker/agent	independent process	execution/lease/artifact	previous compatible artifact
DB	vendor migration	migration history/schema	rollback/forward recovery/restore

21. Architecture 완료 판정 Gate

품질 축	완료 조건
Ownership	단일 Owner, 역방향/순환 의존 없음, Internal 직접 참조 없음
Topology	Local/Remote/MSA/Monolith 에서 업무 계약과 Failure Semantics 동등
Consistency	분산 Local TX 경계, idempotency, UNKNOWN, compensation/reconcile 가 명시
Scale-out	Lease/Fencing/Shared State 로 multi-instance stale writer 차단
Security	Trust boundary, service/user identity, secret/PII, approval/audit 경계가 명확
Operations	ADM/BZA/Gateway/Batch/DB/DR 의 Control/Observation/Recovery path 가 Architecture 와 일치
Evolution	Version/Compatibility/Migration/Rollback 이 Module 경계와 함께 유지